

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Stream III

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

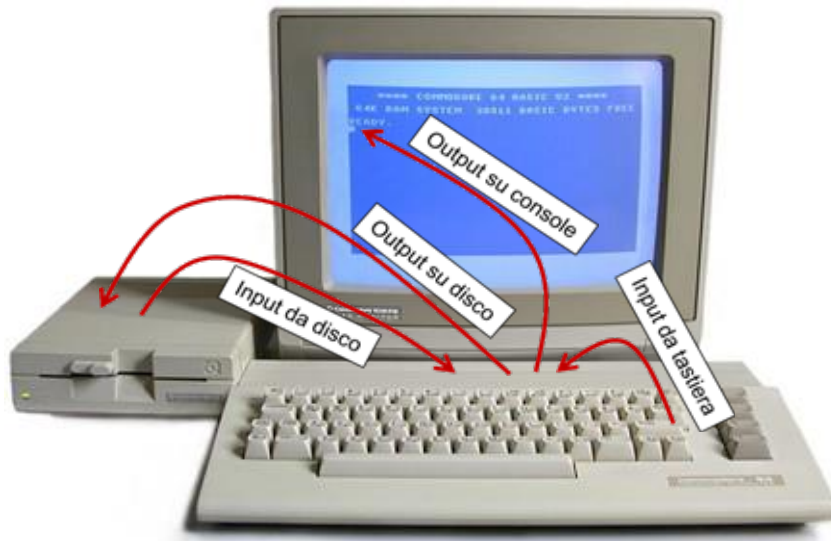
*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Stream<T> parte III

Input & Output

Stream<T>



Gli **Stream** di I/O sono un concetto ben distinto da **Stream API**.

java.util.stream.Collectors: raggruppamento di elementi

- **Collectors.groupingBy**(lambda per una Function)
"lambda" che mappa gli elementi di tipo T in chiavi di tipo S), gli elementi di tipo T vengono collezionati in una List<T>.
Restituisce una **Map**<S, List<T>>
- **Collectors.groupingBy**(lambda per una Function, downstreamCollector),
per raggruppamento multilivello "lambda" che mappa gli elementi di tipo T in chiavi di tipo S, alle chiavi vengono associati valori come specificato dal Collector
"downstreamCollector".
Restituisce una **Map**<S, [come stabilito dal downstreamCollector]>

java.util.stream.Collectors: raggruppamento di elementi

- Ad esempio, per ottenere una (multimappa!) da città a **lista** di persone a partire da una lista di persone:

```
Map<City, List<Person>> peopleByCity =  
    people.stream() //Stream<Person>  
    .collect( Collectors.groupingBy(Person::getCity) );
```

- La stessa multimappa, ma i cui valori siano **insiemi** di persone:

```
Map<City, Set<Person>> peopleByCity=  
    people.stream() //Stream<Person>  
    .collect(  
        Collectors.groupingBy(  
            Person::getCity,  
            Collectors.toSet()  
        )  
    );
```

Collectors.mapping

In raccolte multilivello, per esempio usando `Collectors.groupingBy`, è utile mappare il valore del raggruppamento a qualche altro tipo:

[illegible]



Creare il proprio Collector

- Con il metodo statico `Collector.of` che prende in input 4 argomenti: un `supplier` per creare la rappresentazione interna, l'`accumulator` che aggiorna la rappresentazione con il nuovo elemento, `combiner` che "fonde" due rappresentazioni ottenute in modo parallelo e il `finisher`, chiamato alla fine, che trasforma il tutto nel tipo finale

```
Collector<Input, Accumulator, Output>
```

```
Collector<Person, StringJoiner, String> personNameCollector
```

```
= Collector.of(  
    () -> new StringJoiner(" | "),           // supplier  
    (j, p) -> j.add(p.name.toUpperCase()), // accumulator  
    (j1, j2) -> j1.merge(j2),               // combiner  
    StringJoiner::toString);                 // finisher
```

```
String names = people.stream().collect(personNameCollector);
```

```
System.out.println(names);           // MAX | PETER | PAMELA | DAVID
```

java.util.stream.Collectors: partizionamento di elementi

- `Collectors.partitioningBy(predicato)`: raggruppa in una `Map<Boolean, List<T>>`

// crea una mappa da booleano a lista di interi che soddisfano quel criterio del predicato in input

```
Map<Boolean, List<Integer>> m =  
    l.stream() // Stream<Integer>  
    .collect(Collectors.partitioningBy(x -> x % 2 == 0));
```


Metodi di `java.util.stream.Stream`: `distinct` (intermedia)

- Restituisce un nuovo stream senza ripetizione di elementi (gli elementi sono tutti `distinti` tra loro):

```
List<Integer> l = List.of(3, 4, 5, 3, 4, 1);  
List<Integer> distinti = l.stream() //Stream<Integer>  
    .map(x -> x*x)  
    .distinct()  
    .collect(Collectors.toList());
```

Metodi di `java.util.stream.Stream`: `reduce` (terminale)

- `reduce` è un'operazione terminale che effettua una riduzione sugli elementi dello stream utilizzando la bifunzione data in input
- Vediamo come viene effettuata l'operazione di somma:

```
int somma = 0;  
for (int k : lista)  
    somma += k;
```

- Questa operazione può essere effettuata su uno stream mediante l'operazione `reduce`:

```
lista.stream().reduce(0, (a, b) -> a+b); // oppure:  
lista.stream().reduce(0, Integer::sum);
```

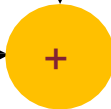
Esecuzione della somma con reduce

Stream<Integer>

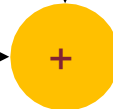
4 8 15 16

reduce(0, (a,b)->a+b)

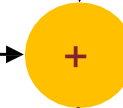
0



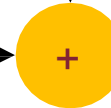
4



12



27



43

Integer

Metodi di `java.util.stream.Stream`: `reduce` (terminale)

- Ad esempio, calcoliamo il prodotto tra interi in una lista:

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8);  
int product = numbers.stream().reduce(1, (a, b) -> a * b);  
// product == 40320
```

- Ad esempio, calcoliamo il massimo tra gli interi di una lista:

```
int max = numbers.stream().reduce(  
    Integer.MIN_VALUE, Integer::max);  
// max == 8
```



Esercizio

- Calcolare la somma del doppio dei valori pari di una lista
- Soluzione con `filter`, `map` e `reduce`:

```
l.stream().filter(e -> e%2 == 0)
        .map(e -> e*2)
        .reduce(0, Integer::sum);
```

- Soluzione con `filter`, `mapToInt` e `IntStream.sum`:

```
l.stream().filter(e -> e%2 == 0)
        .mapToInt(e -> e*2)
        .sum();
```

Metodi di `java.util.stream.Stream`: `limit` (intermedia)

- Limita lo stream a k elementi (k long passato in input)
- Esempio:

```
List<String> elementi = List.of("uno", "due", "tre");
```

```
List<String> reduced = l.stream()
```

```
    .limit(2)
```

```
    .collect(Collectors.toList());
```

```
// contiene ["uno", "due"]
```

Metodi di `java.util.stream.Stream`: `skip` (intermedia)

- Salta `k` elementi (`k long` passato in `input`)
- Esempio:

```
List<String> elementi = List.of("uno", "due", "tre");
```

```
List<String> reduced = l.stream().skip(2).collect(toList());
```

```
// contiene ["tre"]
```

Metodi di `java.util.stream.Stream`: `takeWhile/dropWhile` (intermedie)

- `takeWhile` prende elementi finché si verifica la condizione del predicato
- `dropWhile` salta gli elementi finché si verifica la condizione
- Esempio:

```
List<Integer> elementi = List.of(2, 5, 10, 42, 3, 2, 10);
```

```
List<Integer> reduced = l.stream().takeWhile(x -> x < 42).collect(toList());
```

```
// contiene [2, 5, 10]
```


Metodi di `java.util.stream.Stream`: `anyMatch/allMatch/noneMatch` (terminali)

- Gli stream espongono diverse **operazioni terminali di matching** e restituiscono un booleano relativo all'esito del matching
- Esempio:

```
List<String> l = List.of("albero", "casa", "mondo");
```

```
boolean anyStartsWithA = l.stream().anyMatch(s -> s.startsWith("a"));  
System.out.println(anyStartsWithA); // true
```

```
boolean allStartsWithA = l.stream().allMatch(s -> s.startsWith("a"));  
System.out.println(allStartsWithA); // false
```

```
boolean noneStartsWithZ = l.stream().noneMatch(s -> s.startsWith("z"));  
System.out.println(noneStartsWithZ); // true
```

Metodi di `java.util.stream.Stream`: `findFirst` e `findAny` (terminali)

- Gli stream espongono due operazioni terminali per ottenere il primo (`findFirst`) o un qualsiasi elemento (`findAny`) dello stream
- Esempio:

```
List<String> l2 = Arrays.asList("c", "b", "a");
```

```
// v conterrà il riferimento ad "a"
```

```
Optional<String> v = l2.stream().sorted().findFirst();
```

Metodi di stream: mapToInt e IntStream.summaryStatistics

- E' possibile convertire uno `Stream` in un `IntStream`
- `IntStream` possiede il metodo `summaryStatistics` che restituisce un oggetto di tipo `IntSummaryStatistics` con informazioni su: min, max, media, conteggio

```
List<Integer> p = List.of(2, 3, 4, 5, 6, 7);  
IntSummaryStatistics stats = p.stream().mapToInt(x -> x)  
                                .summaryStatistics();  
  
System.out.println(stats.getMin());  
System.out.println(stats.getMax());  
System.out.println(stats.getAverage());  
System.out.println(stats.getCount());
```

Metodi di stream: flatMap (intermedia)

- `flatMap` permette di "unire" gli stream in un unico stream.
- Sia data `List<String> words=List.of("Java", "Magazine");` vogliamo conteggiare le lettere che occorrono nelle stringhe `Map<String,Long>`.

```
Map<String, Long> letterToCount =  
words.stream()
```

```
    .map(w -> w.split("")) // restituisce uno Stream<String[]>  
    .flatMap(Arrays::stream) // restituisce uno Stream<String>  
    .collect(groupingBy(Function.identity(),  
Collectors.counting()));
```

Esempio di stream di lettere senza ripetizioni

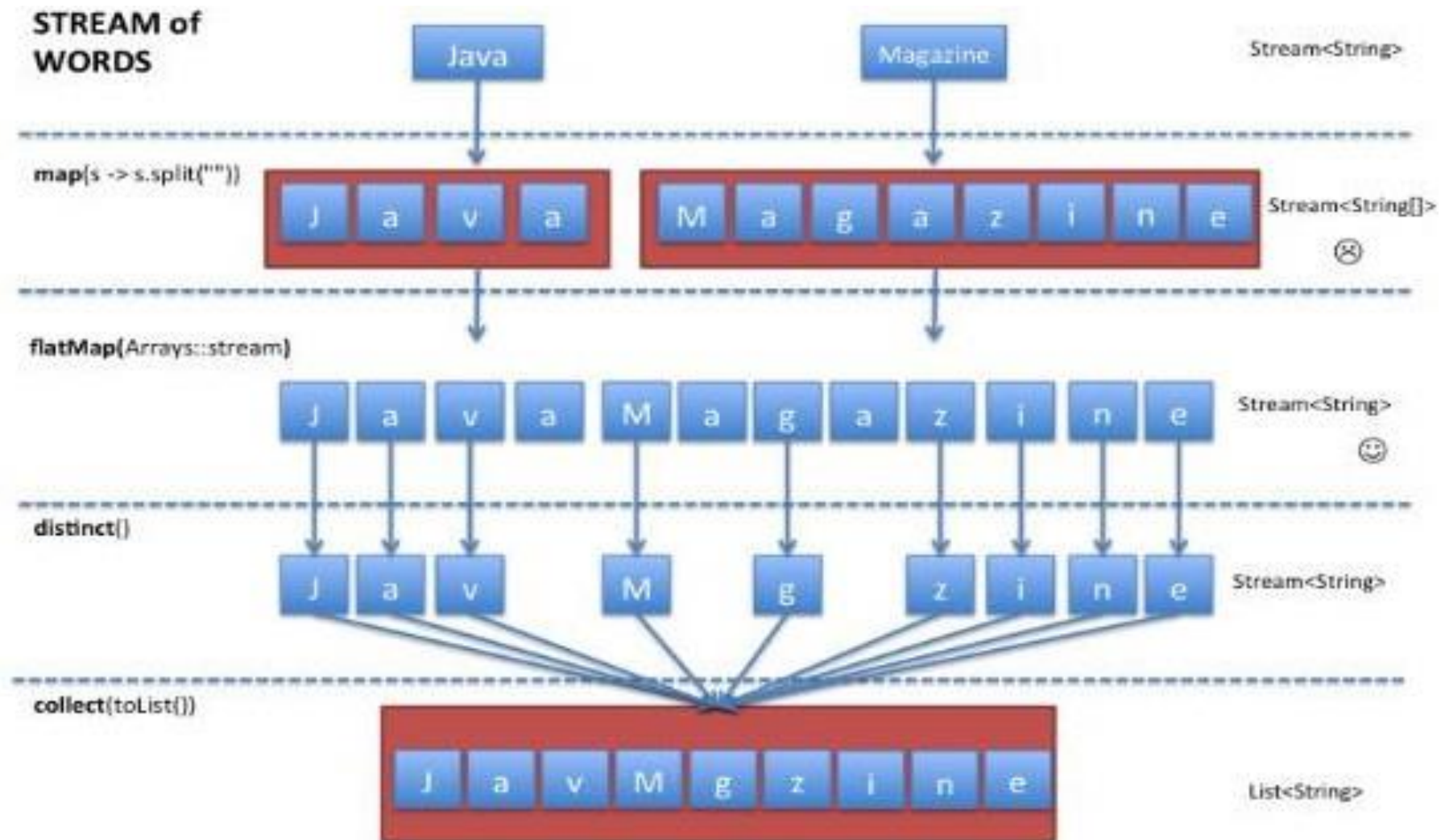
```
List<String> distinctLetters = words.stream() // Stream<String>
```

```
.map(w -> w.split("")) // restituisce uno Stream<String[]>
```

```
.flatMap(Arrays::stream) // restituisce uno Stream<String>
```

```
.distinct() // restituisce uno Stream<String> eliminando le ripetizioni
```

```
.collect(Collectors.toList());
```



Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischì, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.